Advanced Computer Architecture

Chapter #3

Principles of Pipelining & Vector Processing

Ch#3 Principles of Pipelining & Vector Processing

Pipelining, Instruction and Arithmetic Pipelines,

Non-linear pipeline processor

Pipeline processor classification

Arithmetic pipeline design

Instruction pipeline design

Pipeline Hazards

Dynamic instruction scheduling

Ch#3 contd…

Vector instruction types

Vector performance modeling

Vectorization

Vectorizing compiler design

Structures and Algorithms of Array Processors (SIMD Computers)

- SIMD Array Processors
- SIMD Interconnection networks
- Parallel Algorithms for Array Processors $\Rightarrow$ Algorithm examples – matrix multiplication

Pipelining

Pipelining

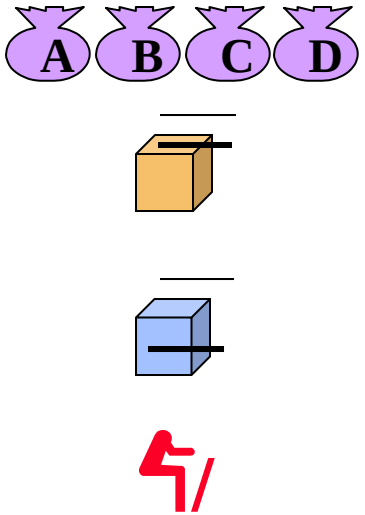
Example

Amar, Basant, Chetana, Devi each have one load of clothes to wash, dry, and fold

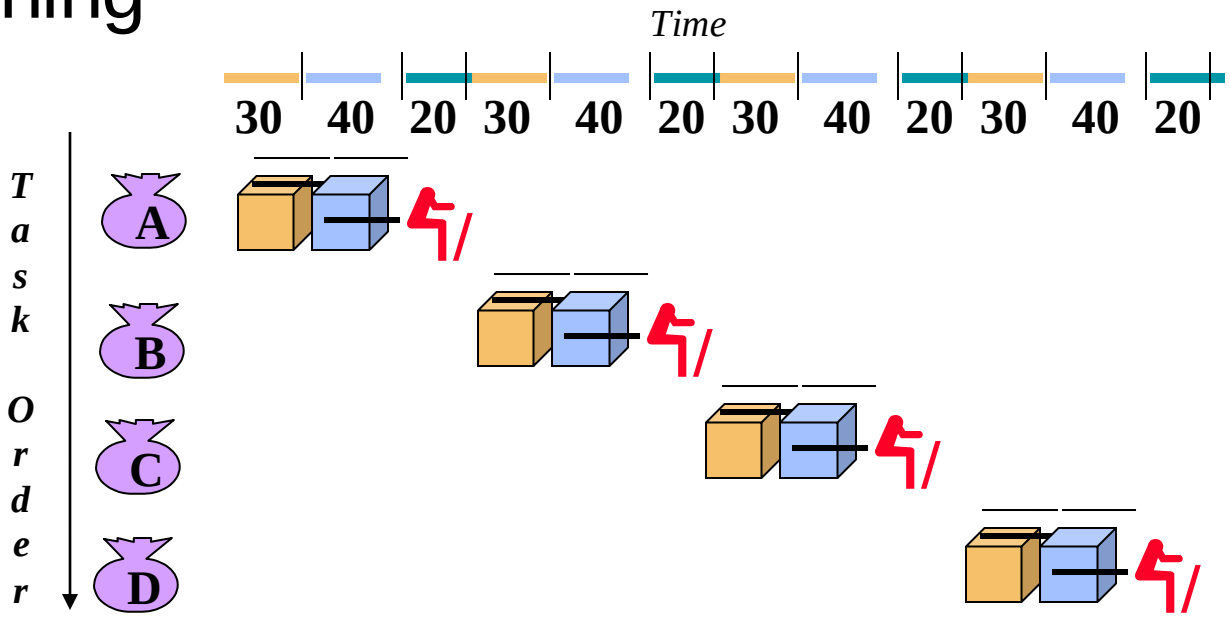
Washer takes 30 minutes

Dryer takes 40 minutes

Folder takes 20 minutes



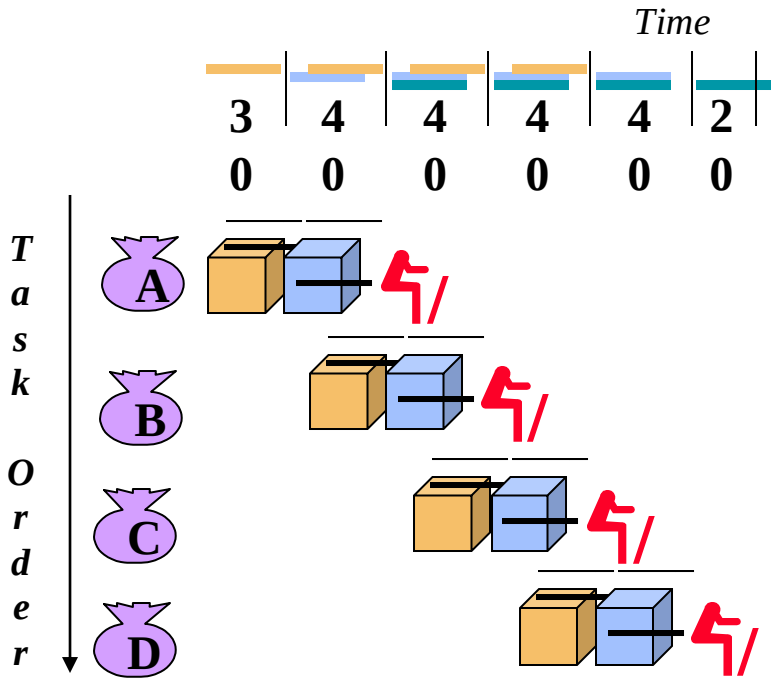
Pipelining



Sequential system takes 6 hours for 4 loads

If pipelining is used, how long would it take?

Pipelining



Pipelined system takes only 3.5 hours for 4 loads

Pipelining

Pipelining doesn't help latency of single task, it helps throughput of entire workload

Pipeline rate limited by slowest pipeline stage

Multiple tasks operating simultaneously

Potential speedup ⇒ Number pipe stages

Unbalanced lengths of pipe stages reduces the speed

Time to "**fill**" pipeline and time to "**drain**" it reduces the speedup

Pipelining

Technique of decomposing a sequential process into sub-operations, with each sub-process being executed in a partial dedicated segment that operates concurrently with all other segments

Technique where multiple instructions are overlapped in the process

Computer pipeline is divided in different stages

Each stage completes a part of an instruction execution in parallel

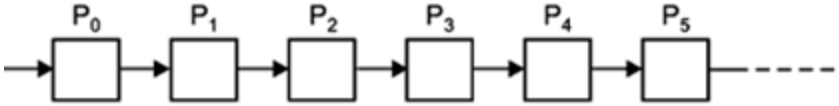
Stages are connected one to the next to form a pipe - instructions enter at one end, progress through the stages, and exit at the other end

Pipelining

Pipelining means realizing temporal parallelism

In pipelining, the problem is divided into a series of tasks that have to be completed one after the other

Each task is executed by a separate unit



The problem is divided into separate functions that must be performed in succession

Pipelining is very much similar to assembly line in an industry ⇒ means an overlapped parallelism

Pipelining

Does not decrease the time for individual instruction execution $\Rightarrow$ but, increases the overall throughput

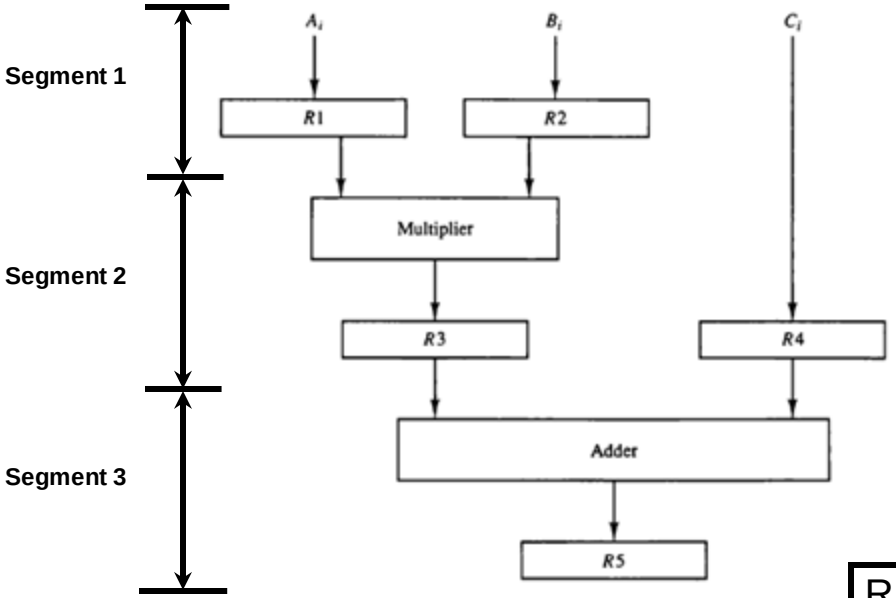
Throughput of instruction pipeline is determined by how often an operation exits the pipeline

Pipe stages are linked together, all the stages must be ready to proceed at the same time

The pipeline designer's goal is to balance the length of each pipeline stage

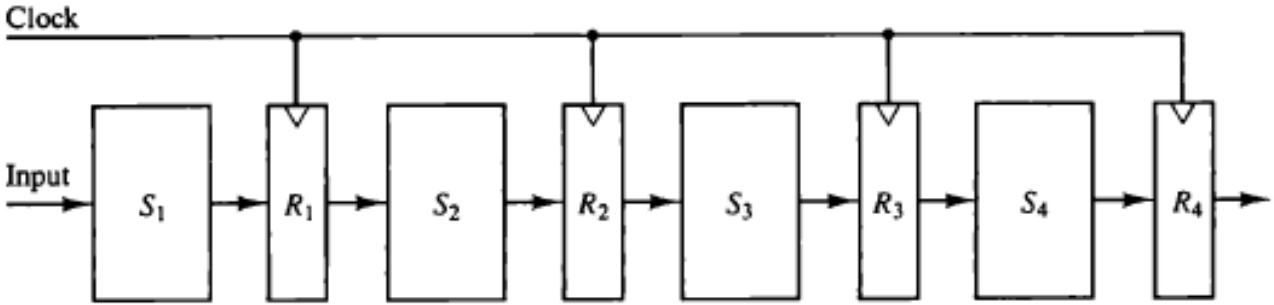
Pipeline Processing

$A_i * B_i + C_i$ for i = 1, 2, 3, ... , 7



Clock Pulse Number	Segment 1		Segment 2		Segment 3
	R1	R2	R3	R4	R5
1	A_1	B_1	—	—	—
2	A_2	B_2	$A_1 * B_1$	C_1	—
3	A_3	B_3	$A_2 * B_2$	C_2	$A_1 * B_1 + C_1$
4	A_4	B_4	$A_3 * B_3$	C_3	$A_2 * B_2 + C_2$
5	A_5	B_5	$A_4 * B_4$	C_4	$A_3 * B_3 + C_3$
6	A_6	B_6	$A_5 * B_5$	C_5	$A_4 * B_4 + C_4$
7	A_7	B_7	$A_6 * B_6$	C_6	$A_5 * B_5 + C_5$
8	—	—	$A_7 * B_7$	C_7	$A_6 * B_6 + C_6$
9	—	—	—	—	$A_7 * B_7 + C_7$

$R_1 \leftarrow A_i$, $R_2 \leftarrow B_i$	Load A_i and B_i
$R_3 \leftarrow R_1 * R_2$, $R_4 \leftarrow C_i$	Multiply and load C_i
$R_5 \leftarrow R_3 + R_4$	Add C_i to product



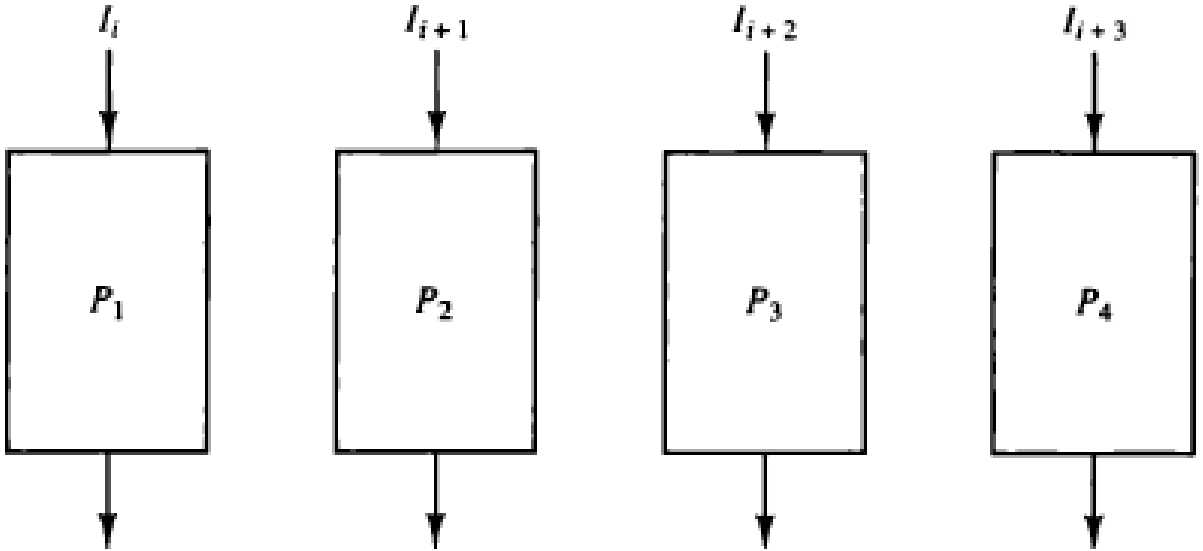
General Structure of a 4-Segment Pipeline

Space-time Diagram for Pipeline:

Segment:	1	2	3	4	5	6	7	8	9
1	T_1	T_2	T_3	T_4	T_5	T_6			
2		T_1	T_2	T_3	T_4	T_5	T_6		
3			T_1	T_2	T_3	T_4	T_5	T_6	
4				T_1	T_2	T_3	T_4	T_5	T_6

Clock cycles

Multiple Functional Units in Parallel



Pipelining

In pipelining different segments may take different times to complete their sub-operation

The clock cycle must be chose to equal time delay of the segment with the maximum propagation time $\Rightarrow$ causes all other segments to waste time while waiting for the next clock

The pipeline technique provides a faster operation over a purely serial sequence even though the maximum theoretical speed is never fully achieved

Can be used for Instruction, Data, Arithmetic etc.

Pipelining

Pipelining is a general-purpose efficiency technique

- It is not specific to processors

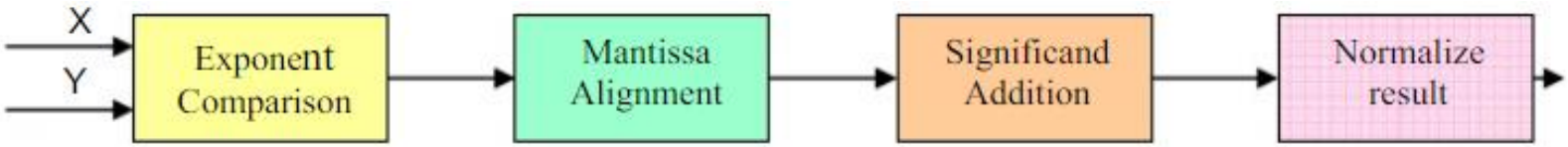
Pipelining is used in many applications such as:

- Factory assembly lines
- Networking
- Server software architecture

Useful to increase throughput in the presence of long latency

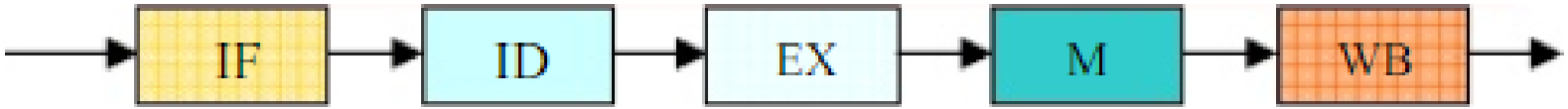
Pipelining

Arithmetic Pipeline:



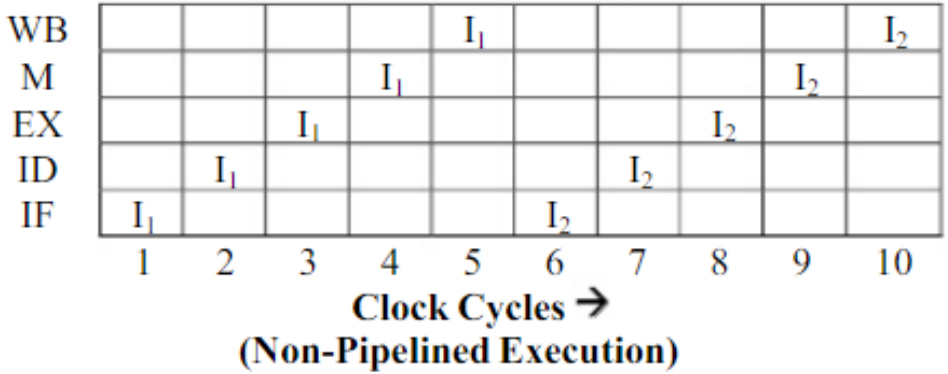
Four stage floating point Adder

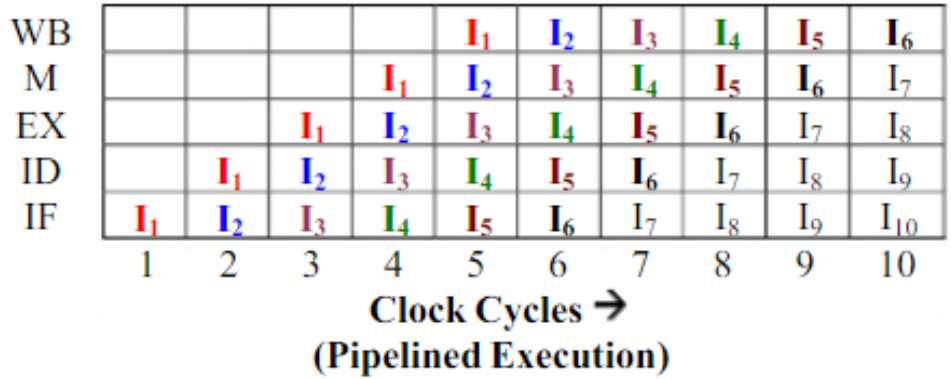
Instruction Pipeline:



Five stage instruction pipeline

Pipelining


WB
M
EX
ID
IF
I1
I2
1 2 3 4 5 6 7 8 9 10
Clock Cycles →
(Non-Pipelined Execution)


WB
M
EX
ID
IF
I1
I2
I3
I4
I5
I6
I7
I8
I9
I10
1 2 3 4 5 6 7 8 9 10
Clock Cycles →
(Pipelined Execution)

Pipeline Principles and Implementation

Pipeline Principles and Implementation

Pipeline processor ⇒ cascade of processing stages ⇒ connected to perform a fixed function over a stream of data flowing from one end to another

Pipelining involves:

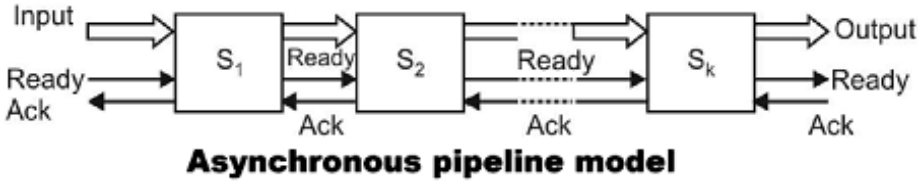
- The input task to be divided into sequence of subtasks
- Each subtask can be executed by a specialized hardware that operates concurrently with other pipeline stages
- There is a constant stream of tasks into the pipe and there is overlapped execution at the subtask level

Linear Pipeline Processor

Asynchronous Model

Synchronous Model

Asynchronous Model



Asynchronous pipeline model

In asynchronous pipeline, the data flow between the adjacent stages is controlled by handshaking protocol

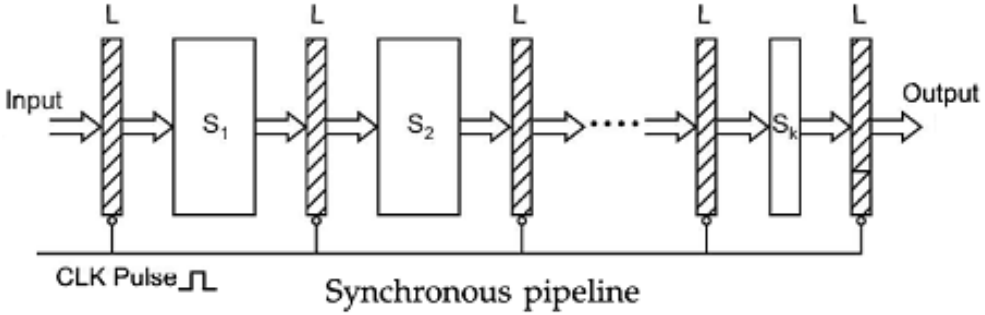
When the stage, S_i, is ready to transmit, it sends a ready signal to next stage S_{i+1}

After stage, S_{i+1}, receives the incoming data, it returns an acknowledgement signal to previous stage S_i

This model is useful in designing the communication channels in the message - passing multi-computers where pipeline worm whole routing is practiced

May have a variable throughput rate ⇒ different amounts of delay might be in the different stages

Synchronous Model



Consists of a cascade of processing stages

Stages are combinational circuits performing arithmetic or logic operations over the data stream flowing through the pipe

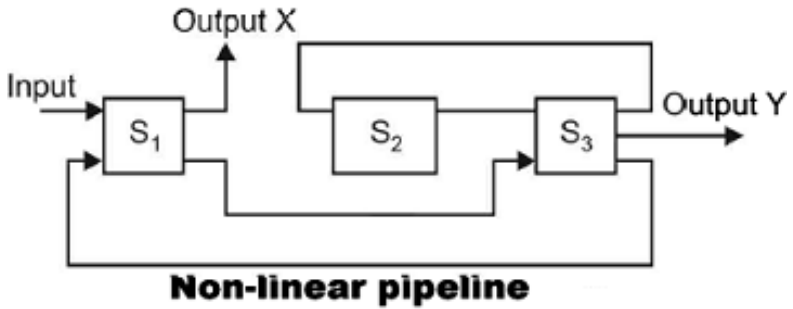
Stages are separated by high-speed interface latches

Latches are fast-register for holding the intermediate results between the stages

Information-flows between adjacent stages are under the control of common clock (CLK) applied to all the latches simultaneously

Non-Linear Pipeline Processor

Non-Linear Pipeline Processor



Allows feed-forward and feedback connections in additions to the streamline connections ⇒ non-linear pipeline

A dynamic pipeline is sometimes called as a non-linear pipeline because of feed forward and feedback connection

It can be reconfigured to perform different functions at different times

The linear pipelines are static pipelines ⇒ as they perform fixed functions

Figure shows a non-linear pipeline that can perform two functions X and Y

Pipeline Processor Classifications

Based on levels of processing

Based on Pipeline Configuration

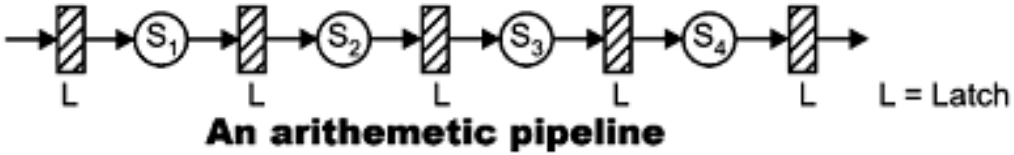
Handler's Classification - Based on levels of processing

Classification according to the levels of processing ⇒ proposed by Handler

- Arithmetic Pipelining
- Instruction Pipelining
- Processor Pipelining

Arithmetic Pipelining

The ALUs of computer are segmented in various data formats for pipeline operations



An arithemetic pipeline

For example:

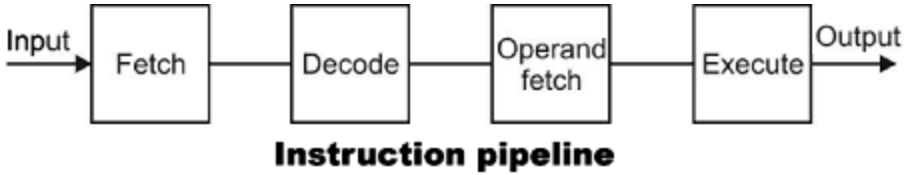
- 4 - stage pipes in Star 100
- 14 - stage pipes in Cray-1
- 26 - stage pipes in Cyber-305

Instruction Pipelining

Also known as instruction look ahead

The execution of stream of instructions is pipelined by overlapping the execution of current instruction with fetch, decode and operand fetch of subsequent instructions

Almost all high performance computers have instruction execution pipeline



Instruction pipeline

Processor Pipelining

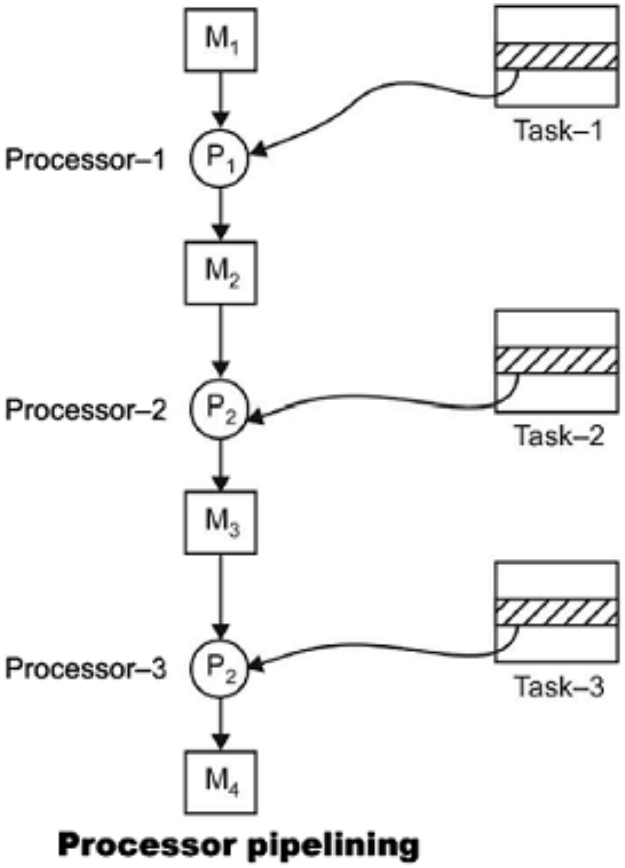
Same data stream is processed by the cascade of processors

Each processor performs a specific task

The data stream passes the first processor with results stored in a memory block which is also accessible by the second processor

The second processor processes this result and passes it to third and so on

This pipeline of multiple processor is not much popular



Processor pipelining

Based on Pipeline Configuration

Based on Pipeline Configuration - Ramamoorthy and Li's Classification

- Uni-function vs. Multifunction Pipeline
- Static vs. Dynamic Pipeline
- Scalar vs. Vector Pipeline

Uni-function vs. Multifunction Pipeline

Uni-function Pipeline ⇒ a pipeline with fixed and dedicated function, e.g. floating-point adder

Multifunction Pipeline ⇒ a pipeline that performs different functions either at different times or at the same time by interconnecting different subsets of stages in the pipeline, e.g. TI-ASC has 4 multifunction pipeline processors

TI-ASC: Advanced Scientific Computer ⇒ supercomputer designed & manufactured by Texas Instruments

Static vs. Dynamic Pipeline

Static Pipeline ⇒

- Assumes only one functional configuration at a time
- Can be either unifunctional or multifunctional
- Function of this pipeline should not change frequently
- Frequent change in the function will lead to degradation of performance

Dynamic Pipeline ⇒

- Several functional configurations can exist simultaneously
- Must be multifunctional
- The control in dynamic pipe is complex as compared to that in static

Unifunctional pipeline must be static

Unifunctional or multifunctional static pipelines are more popular

Scalar vs. Vector Pipelines

This classification is based on the instruction or data types

Scalar Pipeline ⇒ processes scalar operands under the control of Do-loop

- Instructions in a small Do-loop are normally prefetched into instruction buffer
- The scalar operands which are required for repeated scalar instruction are moved into a data cache ⇒ ensures the continuous supply of operands to the pipeline, e.g. IBM-360

Vector Pipeline ⇒ processes vector instructions over vector operands

- Computers having vector instructions ⇒ vector-processors, e.g., STAR-100, Cyber-250, Cray-1

Scalar pipelines are under software control whereas the vector pipeline uses firmware and hardware controls for processing vector operands

Arithmetic Pipeline Design

Used to speed-up numerical computations

Computer Arithmetic

In computers, arithmetic is performed with finite precision due to use of fixed-size memory words or registers

Fixed-point or integer arithmetic offers a fixed range of numbers during operations

Floating-point arithmetic operates over the large dynamic range of numbers

In modern computers, both fixed-point and floating-point arithmetic operations are performed by separate hardware

Finite precision means that the number exceeding the limit must be truncated or rounded to provide a precision within the number of significant bits allowed

In case of floating-point numbers, exceeding the exponent range means error conditions called as overflow or underflow

Fixed-Point Operations

Different notation for fixed-point operations

- Signed magnitude
- One's complement
- Two's complement

The two's complement notation is the more popular

Most of the computers are using this because of its unique representation of all numbers including zero

Addition, subtraction, multiplication and division are the basic arithmetic operations

Floating-Point Operations

There are four basic arithmetic operations for a pair of floating point numbers

Say x = (m_x, e_x) and y = (m_y, e_y) are two floating point numbers

Where mx and my are their mantissa and e_x and e_y are exponents

For simplicity and clarity, say $e_x \leq e_y$ and base r = 2

Then the four basic operations on x and y will be:

$$x + y = (m_x \times 2^{e_x - e_y} + m_y) \times x^{e_y} \qquad x - y = (m_x \times 2^{e_x - e_y} - m_y) \times x^{e_y}$$

$$x \times y = (m_x \times m_y) \times 2^{e_x + e_y} \qquad x \div y = (m_x \div m_y) \times 2^{e_x - e_y}$$

These operations can be divided into two groups-one for the exponent operations and other for mantissa operations ⇒ these operations demand double hardware for floating point unit as compared to that required in a fixed point unit

Elementary Functions

Include trigonometric, exponential, logarithmic and other transcendental functions

Truncated polynomials or power series can be used to evaluate the elementary functions such as sin x, ln x, e^x, cosh x, $\tan^{-1}$, x^3 etc.

The computer arithmetic can be implemented by hardwired random logic circuitry as well as by table lookup using ROM or RAM in the memory

Table lookups can be used to see the values of frequently used constants

Arithmetic Pipeline Stages

Depending on the arithmetic operations (add, subtract, multiply, divide, square, square root, logarithm etc) which is to be implemented, different pipeline stages in an arithmetic unit require different hardware logic

Basically, we need some hardware for add and shift operations only since all operations involve these two operations only

Arithmetic and logic shifts can be easily implemented with shift registers

Arithmetic Pipeline Stages

For implementing the arithmetic pipelines we generally use two types of adders

High speed operations requires the use of carry-propagation adder (**CPA**) which adds two numbers and produces an arithmetic sum or it requires a carry-save adder (**CSA**) to add three input numbers and produce one sum output and a carry output

Carry propagation adder (CPA): adds two numbers such that carries generated in successive digits are propagated

Carry save adder (CSA): adds two numbers such that carries generated are not propagated rather these are saved in a carry vector

Arithmetic Pipeline Design Examples

CPA and CSA can be used to design a multiply pipeline and a divide pipeline

Multiply Pipeline Design

A number of adders are used for multiple-number addition

Example of multiplication of two 6-bit numbers

Stage 1: Generate all $6 \times 6 = 36$ immediate product terms

Stage 2: Six numbers are added by forming two groups, each group having three numbers to get two sum vectors and two carry vectors $\Rightarrow$ CSAs are used

Stage 3: The four results generated by stage 2 are added using another CSA

Stage 4: CSA adds all the three vectors received and generate on sum and one carry vector

Stage 5: Adds the sum and carry vector using a CPA

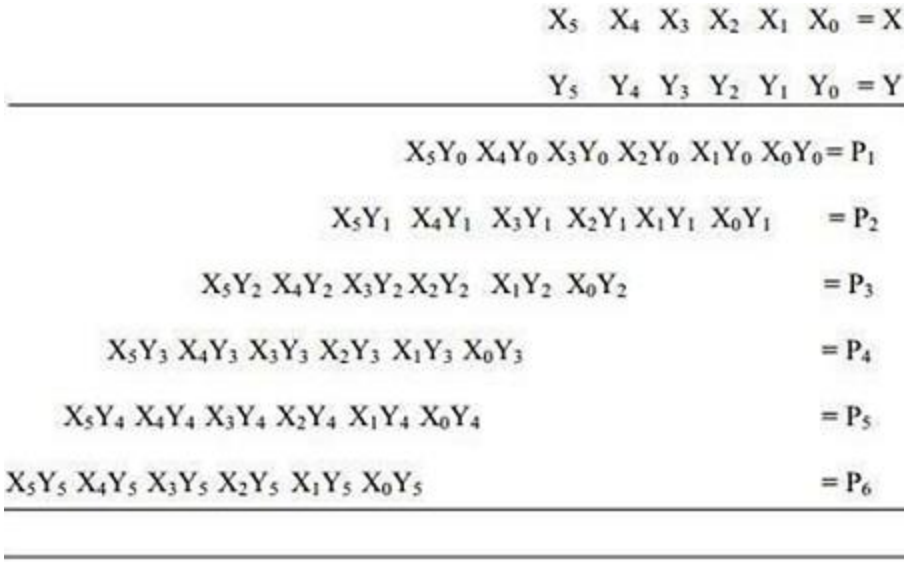
Fixed Arithmetic pipeline

Let us see an example of multiplication of fixed numbers

Two fixed-point numbers are added by the ALU using add and shift operations

This sequential execution makes the multiplication a slow process

The process of adding the multiple copies of shifted multiplicands is shown as ⇒



Identifying the stages for pipeline

First Stage $\Rightarrow$ generates the partial product of the numbers, which form the six rows of shifted multiplicands

Second Stage $\Rightarrow$ six numbers are given to the two CSAs merging into four numbers

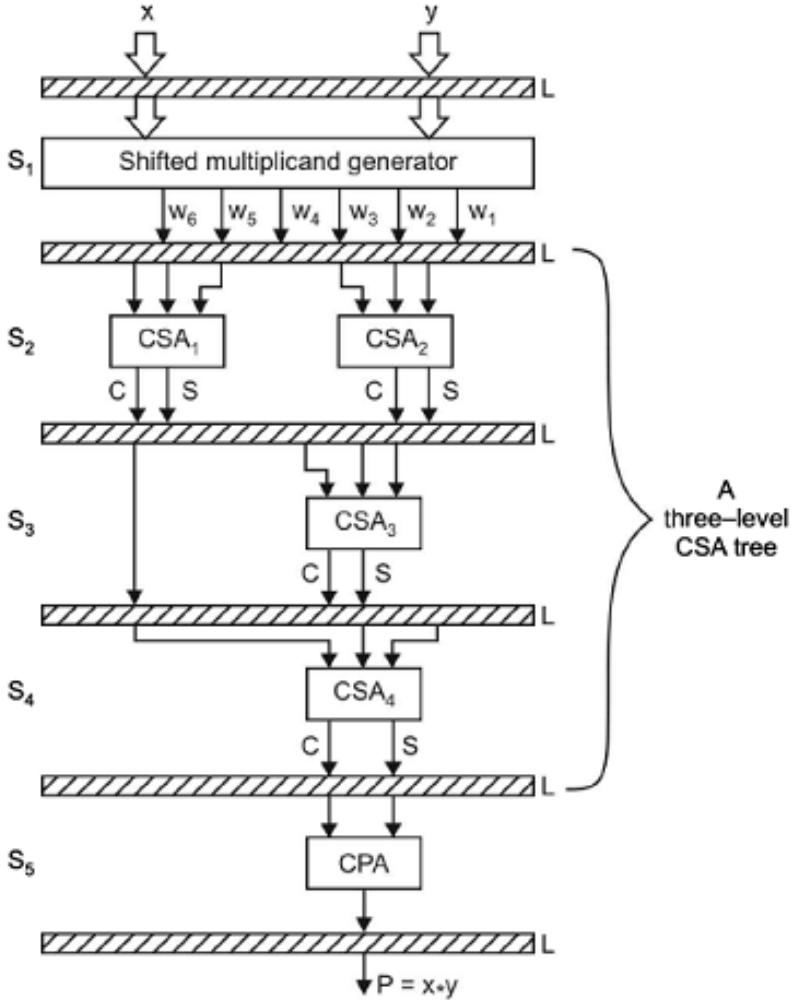
Third Stage $\Rightarrow$ CSA merging 3 numbers

Fourth Stage $\Rightarrow$ a single number merging three numbers into 2 numbers

Fifth Stage $\Rightarrow$ last two numbers are added through a CPA to get the final product

- Totally 4 CSAs are interconnected to form 3 level CSA tree
- CSA tree merges 6 numbers into two $\Rightarrow$ sum vector S and carry vector C
- Final stage CPA adds sum vector S and carry vector C to produce the product P
- The given CSA tree that adds only multiple single bit numbers $\Rightarrow$ bit-slice Wallace tree

A Pipelined 6-bit Multiplier



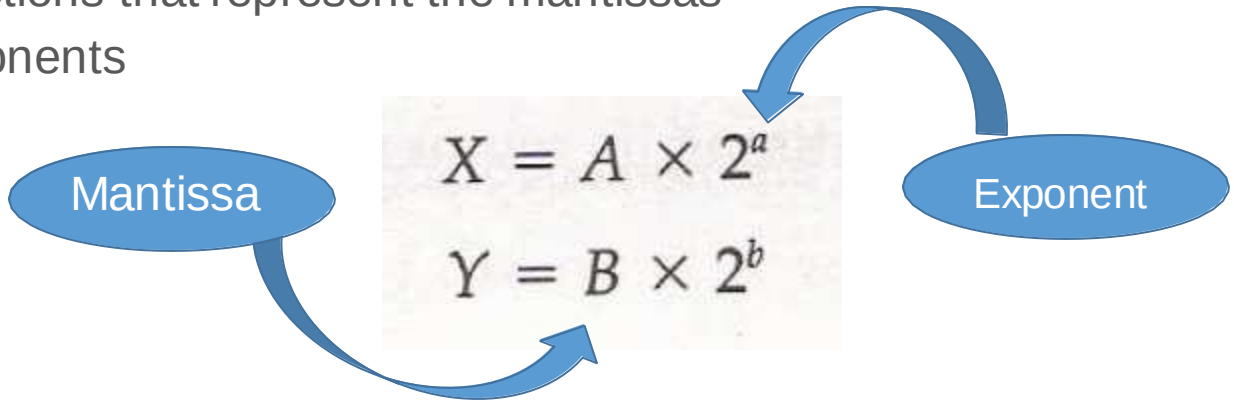
Floating point operations

The inputs to floating point adder pipeline are two normalized floating point numbers

Floating-point operations can be decomposed into sub operations

Example: Let $X = A \times 2^a$ and $Y = B \times 2^b$ are two inputs

- A and B are two fractions that represent the mantissas
- a and b are the exponents



Arithmetic Pipelining with four Sub-operations

Compare the exponents ⇒ the larger exponent is chosen as the exponent of the result

Align the mantissas ⇒ the exponent difference determines how many times the mantissa associated with the smaller exponent must be shifted

Add or subtract the mantissas

Normalize the result

- When an overflow occurs, the mantissa of the sum or difference is shifted right and the exponent incremented by one
- If an underflow occurs, the number of leading zeros in the mantissa determines the number of left shifts in the mantissa and the number that must be subtracted from the exponent

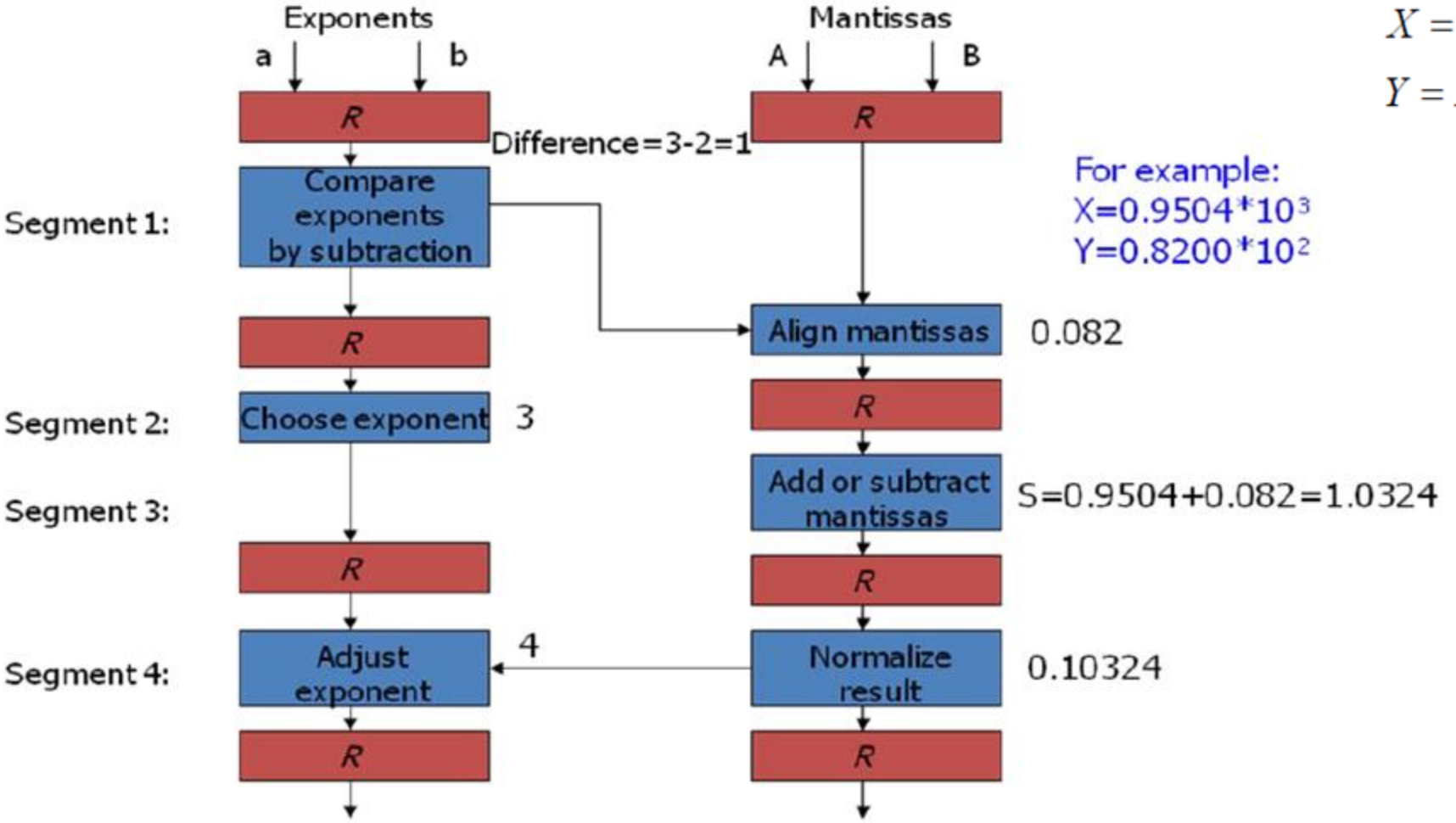


Fig Pipeline for floating point addition and subtraction

Division Pipeline Design

Left as an exercise

Instruction Pipeline Design

Instruction Pipeline Design

A stream of instructions can be executed by a pipeline in an overlapped manner

A typical instruction execution consists of a sequence of operations such as:



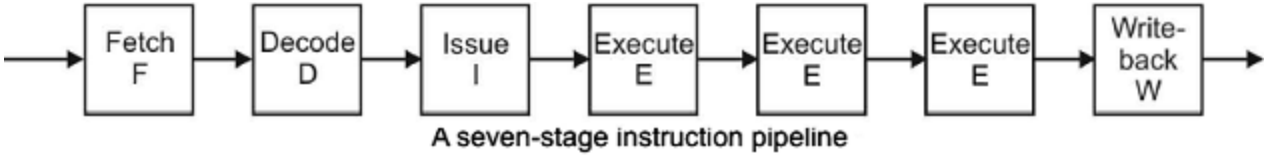
Instruction Execution Phases

These phases are ideal for overlapped execution on a linear pipeline

Each phase may require one or more clock cycles to execute, depending on the instruction type and processor/memory architecture

Pipelined Instruction Processing

A typical instruction pipeline is shown as:



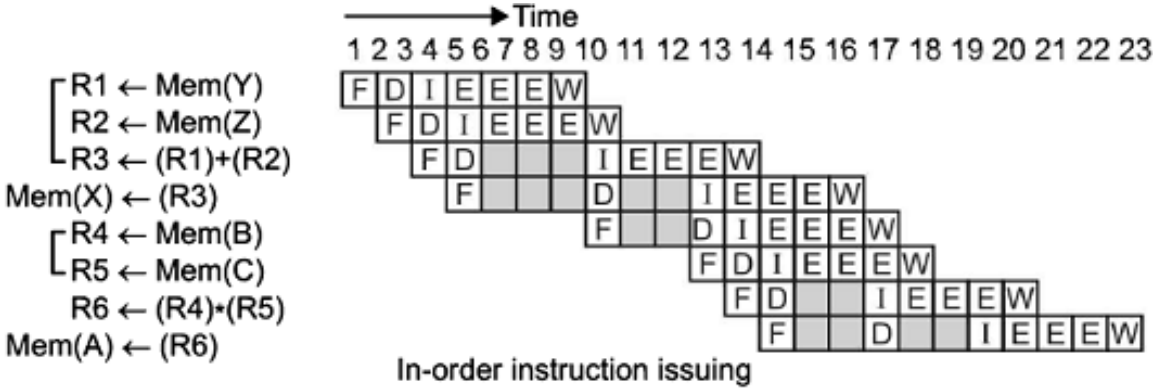
A seven-stage instruction pipeline

Fetch stage (F) ⇒ fetches instruction from a cache memory (one per cycle)

Decode stage (D) ⇒ knows the instruction function to be performed to identify the resources needed; Resources include general-purpose registers, buses and functional units

Issue stage (I) ⇒ reserves resources; pipeline control interlocks are maintained

The operands are also read from registers during the issue stage

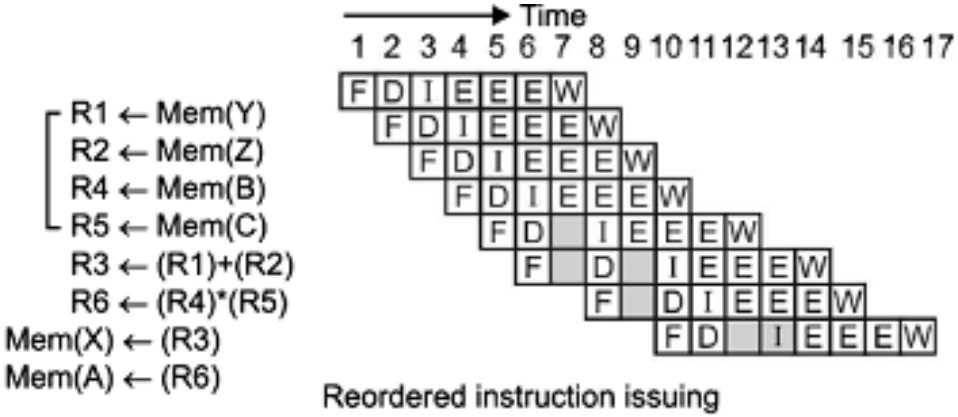


Instructions are executed in one or several execute stages (E)

The last writeback stage (W) is used to write results into the registers

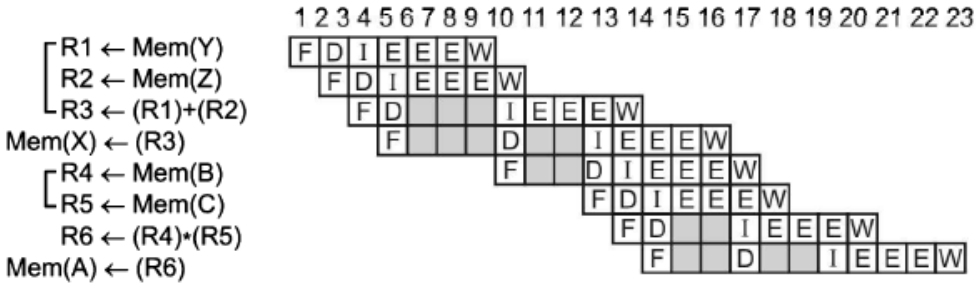
Memory load or store operations are treated as a part of execution

Assume load and store instructions take four execution clock cycles, while floating-point add and multiply operations take three cycles



Pipelined execution of X = Y + Z and A = B * C

Analysis



First figure illustrates the issue of instructions following the original program order

The shaded latency or conflicts or due to data dependencies

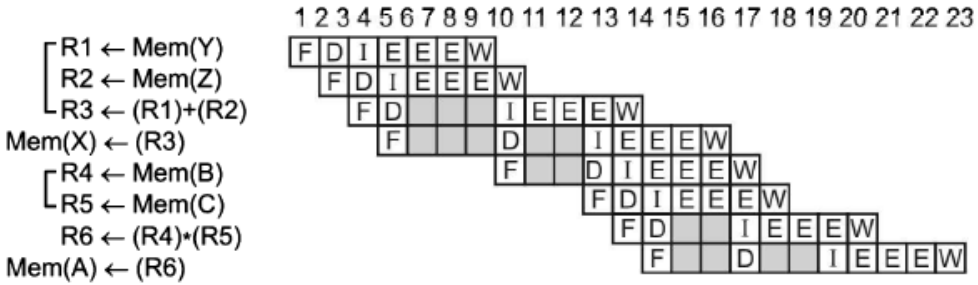
The first two load instructions issue on consecutive cycles

The add is dependent on both loads and must wait three cycles before the data are loaded in

Similarly the store of the sum to memory location X must wait three cycles for the add to finish due to a flow dependence

Similar blockages are there during the calculation of A

Analysis

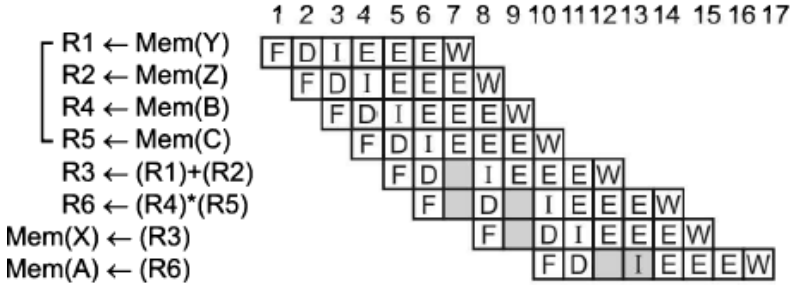


Second figure shows an improved timing after the instruction issuing order is changed to eliminate unnecessary delays due to dependence

Idea is to issue all four load operations in the beginning

Both the add and multiply instructions are blocked fewer cycles due to this data prefetching

Reordering should not change the end results



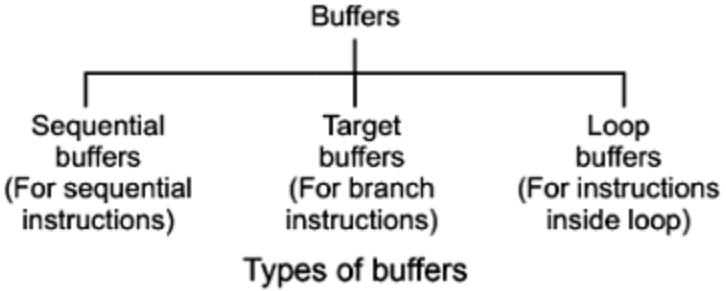
Mechanism for Instruction Pipelining

Prefetch Buffers

Multiple functional units

Internal data forwarding and Register tagging

Prefetch Buffers



Types of buffers

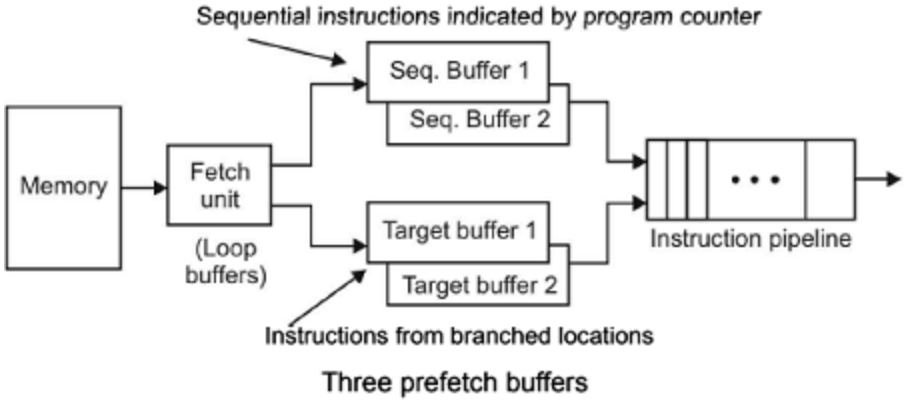
In one memory-access time, a block of consecutive instructions are fetched into a prefetch buffer

Different types of buffers are: Sequential, Target & Loop buffers wich each having own role

Sequential and target buffers operate in a FIFO order

Buffers become part of the pipeline as additional stages

Conditional branch instruction cause both sequential and target buffers to fill with instructions $\Rightarrow$ two buffers work alternately to prevent a collision between the instructions flowing into and out of the pipeline



Three prefetch buffers

The loop buffer is maintained by the fetch stage of the pipeline

The loop buffer operates in two steps:

- Contains instructions sequentially ahead of the current instruction ⇒ saves instruction fetch time from memory
- Recognizes when the target of a branch falls within the loop boundary ⇒ advantages of this scheme ⇒ unnecessary memory accesses can be avoided

Multiple functional units

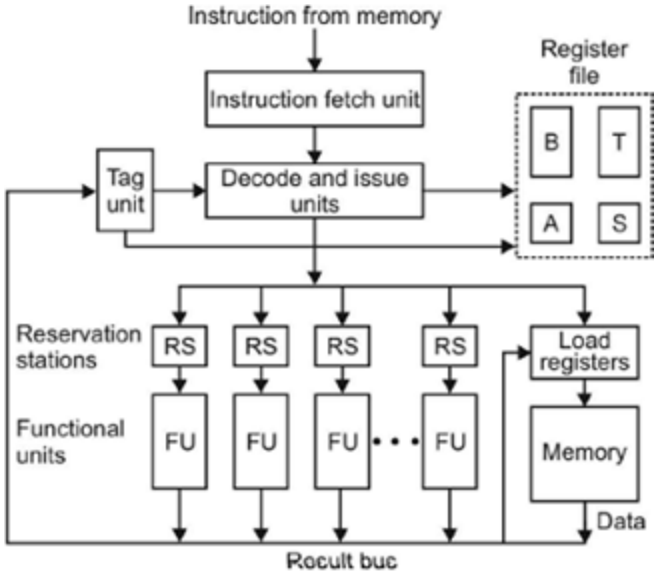
Certain pipeline stage can be the bottleneck

Corresponds to the row with the maximum number of checkmarks in the reservation table

⇒ This bottleneck problems can be addressed by using multiple copies of the same stage simultaneously

This leads to the use of multiple execution units in a pipelined processor design as shown ⇒

Multiple functional units are supposed to operated in parallel, once the dependencies are resolved



Internal data forwarding and Register tagging

The techniques used for instruction pipelining

- Internal forwarding
- Register tagging

Internal Forwarding

A short-circuit technique for replacing unnecessary memory accesses by register-to-register transfers in a sequence of fetch-arithmetic-store operations (as register access is faster than memory access)

Types:

- Store-fetch forwarding
- Fetch-fetch forwarding
- Store-store overwriting

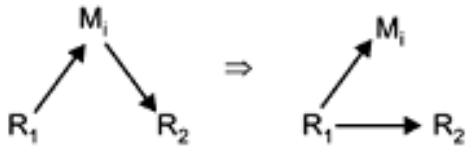
Store-Fetch Forwarding

Sequence of the two operations store then fetch can be replaced by two parallel operations, one store and one register transfer

$$\left.\begin{array}{l} M_i \leftarrow (R_i)\ \text{(store)} \\ M_2 \leftarrow (M_i)\ \text{(fetch)} \end{array}\right\} \text{2 memory accesses}$$

being replaced by

$$\left.\begin{array}{l} M_i \leftarrow (R_1)\ \text{(store)} \\ R_2 \leftarrow (R_1)\ \text{(reg. transfer)} \end{array}\right\} \text{1 memory access}$$



Fetch-Fetch Forwarding

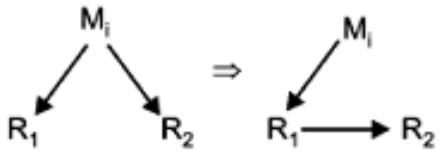
Two fetch operations can be replaced by one fetch and one register transfer

⇒ One memory access has been eliminated

$$\left.\begin{array}{l} R_i \leftarrow (M_i)\ \text{(fetch)} \\ R_2 \leftarrow (M_i)\ \text{(fetch)} \end{array}\right\} \text{2 memory accesses}$$

Being replaced by

$$\left.\begin{array}{l} R_1 \leftarrow (M_1)\ \text{(fetch)} \\ R_2 \leftarrow (R_1)\ \text{(reg. transfer)} \end{array}\right\} \text{1 memory access}$$



Store-Store Overwriting

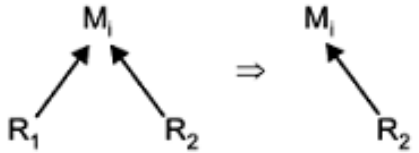
Two memory updates (stores) of the same word can be combined into one

As the second store overwrites the first, it can be combined

$$\left.\begin{array}{l} M_i \leftarrow (R_1)\ \text{(store)} \\ M_i \leftarrow (R_2)\ \text{(store)} \end{array}\right\} \text{2 memory accesses}$$

Being replaced by

$$\left.\begin{array}{l} M_i \leftarrow (R_2)\ \text{(store)} \end{array}\right\} \text{1 memory access}$$



Register Tagging

Refers to the use of tagged registers, buffers and reservation stations for exploiting concurrent activities among multiple arithmetic units

May be used in almost any computer that has multiple functional pipelines and accumulators

Example: Let the inner loop of a certain program is completed to perform the following operations in a sequence:

1. $R_0 \leftarrow (M_1)$ *(fetch)*
2. $R_0 \leftarrow (R_0) + (M_2)$ *(add)*
3. $R_0 \leftarrow (R_0) \times (M_3)$ *(mul)*
4. $M_4 \leftarrow R_0$ *(store)*

Here, the internal-forwarding technique can be applied to these sequence of executions with minimum number of memory references

Pipelining Hazards

Pipelining Hazards

Caused by resource-usage conflicts among various instructions in the pipeline ⇒ triggered by inter-instruction dependencies

When successive instructions overlap their fetch, decode and execution through a pipeline-processor, inter-instruction dependencies may arise to prevent the sequential data flow in the pipeline

An instruction may depend on the results of the previous instruction ⇒ until the completion of previous instruction the current instruction cannot be initiated

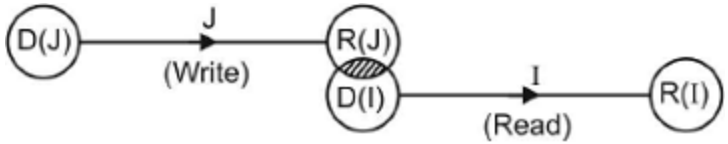
Similarly, two stages of the pipeline may need to update the same memory location

These kinds of hazards if not properly detected and resolved, could result in an interlock-situation in the pipeline or produce unreliable results

Data Hazards

Different types of data dependent hazards:

Write After Read (WAR) Hazards: may occur when J-attempts to modify some data object that is read by I



As it is modified by J, so J writes

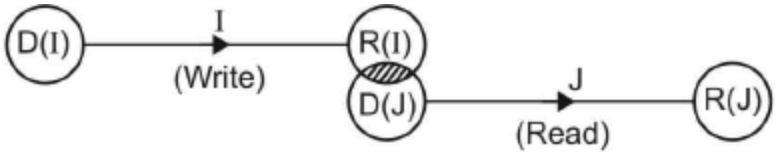
The necessary condition for this hazard are

$$D(I) \cap R(J) \neq \phi \text{ for WAR}$$

$$\uparrow$$

$$(f_i) \text{ or null}$$

Read-After-Write (RAW) Hazards: may occur between two instructions I & J when J attempts to read some data object that has been modified by I

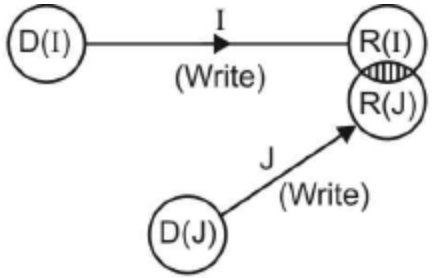


As modified by I, I writes

The necessary condition for this hazard is:

$$R(I) \cap D(J) \neq \phi \text{ for RAW}$$
(null)

Write-After-Write (WAW) Hazards: may occur if both I and J attempt to modify the same data object



As both I & J modify, so both write

The necessary condition for this hazard is

$$R(I) \cap R(J) \neq \phi \text{ for WAW}$$
$$\uparrow$$
$$(f_i) \text{ or null}$$

Note: RAR is not there because when we do read-after-read nothing is changed so RAR is not a problem

Control Hazards

Resource-objects are used to refer to working-registers, memory-locations & special flags

Resource-object ⇒ data-object

Each instruction can be considered as a mapping from a set of data objects to a set of other data objects

Domain, D(I) of an instruction I is defined as the set of 'resource-objects' whose contents may affect the execution of I (i.e., operands only)

Range, R(I) of an instruction, I is the set of resource objects whose data objects may be modified by execution of instruction I (i.e., results of execution of instruction)

contd…

The operands to be used in an instruction execution are retrieved (read) from its domain and the results will be stored (written) in its range

Let the execution of two instructions I & J in a program

Instruction J appears after instruction I in the program

There may be none or other instructions between instruction I & J

Instruction J may enter the execution pipe before or after the completion of the execution of instruction I

The improper timing & data dependencies may create some hazard situations

Hazard Detection

Can be done in the instruction fetch stage of a pipeline by comparing the domain and range of incoming instruction with those of the instruction being processed, e.g. when any one of the conditions WAR or RAW or WAW is detected a warning signal can be generated to prevent the hazard from taking place

Another approach is to allow the incoming instruction through the pipeline and distribute the detection to all potential pipeline stages ⇒ offers better flexibility at the expense of increased hardware control

Hazard Resolution

Once a hazard is detected ⇒ system should resolve it

Let an instruction sequence {...I, I+1, ...J, J+1, ...} in which a hazard has been detected between the current instruction J and a previous instruction I

A straightforward approach is to stop the pipe and suspend the execution of instruction J, J+1, J+2 ... until the instruction I has passed the point of resource conflict

To avoid RAW-type of hazards, IBM engineers developed a short-circuiting approach ⇒ called as data-forwarding, which forwards multiple copies of the data to as many waiting instruction as may wish to read it

contd…

Another type of hazard $\Rightarrow$ due to the job-scheduling problem

Once a task is initiated in a static pipeline, its flow pattern is fixed

An initiation refers to the start of a single function evaluation when two or more initiations attempt to use the same stage at the same time, a collision results

The job sequencing problem is "**To properly schedule queued tasks awaiting initiation in order to avoid collisions and to achieve better throughput**“

Problems based on pipeline hazards

Consider the following instruction sequence to identify hazards

$I_1 : R_1 \leftarrow (M(R_4))$; *Load*
$I_2 : R_2 \leftarrow R_1 + R_2$; *Add*
$I_3 : M[R] \leftarrow R_2$; *Store*
$I_4 : R_2 \leftarrow R_6$ *and* R_3 ; *AND (logical)*
$I_5 : R_6 \leftarrow R_6 + 1$; *Increment ?*

I_1, I_2 ⇒ RAW hazard
I_2, I_4 ⇒ WAW hazard
I_4, I_5 ⇒ WAR hazard

Identify the hazards in following instruction stream

$$I_1 : R_2 \leftarrow R_2 + R_3$$
$$I_2 : \textit{if zero } (R_2) \textit{ then goto } L3 \ ?$$

I_1, I_2 ⇒ show RAW hazard

Data dependency hazards are problematic

They may prevent simultaneous processing of two instructions in the pipeline ⇒ reduced throughput

Dynamic Instruction Scheduling

Dynamic Instruction Scheduling

Different approaches:

- **Static Scheduling Approach**
- **Register Scoreboarding Approach**
- **Tomasulo's Approach**

A method in which the hardware determines which instructions to execute, as opposed to a statically scheduled machine, in which the compiler determines the order of execution

Different methods for scheduling instructions through an instruction pipeline

The static scheduling scheme is supported by an optimizing compiler

Dynamic scheduling is achieved using a technique such as Tomasulo's register-tagging scheme build in the IBM 360/91, or the scoreboarding scheme build in the CDC 6600 processor

Static Scheduling

Stage delay:	Instruction:		
2 cycles	Add	R0, R1	/R0 ← (R0) + (R1)/
1 cycle	Move	R1, R5	/R1 ← (R5)/
2 cycles	Load	R2, M(α)	/R2 ← (Memory (α))/
2 cycles	Load	R3, M(β)	/R3 ← (Memory (β))/
3 cycles	Multiply	R2, R3	/R2 ← (R2) × (R3)/

Data dependencies in a sequence of instructions $\Rightarrow$ interlocked relationships among them

Interlocking $\Rightarrow$ resolved through a compiler-based static scheduling approach

A compiler or a post processor can be used to increase the separation between interlocked interactions

In the given code segment

- The multiply instruction cannot be initiated until the preceding load is complete
- Data dependence will stall the pipeline, as the two loads overlap by one cycle

Static Scheduling

Stage delay:	Instruction:		
2 cycles	Add	R0, R1	/R0 ← (R0) + (R1)/
1 cycle	Move	R1, R5	/R1 ← (R5)/
2 cycles	Load	R2, M(α)	/R2 ← (Memory (α))/
2 cycles	Load	R3, M(β)	/R3 ← (Memory (β))/
3 cycles	Multiply	R2, R3	/R2 ← (R2) × (R3)/

Two loads, can be moved ahead to increase the spacing between them and the multiply instruction (as they are independent of the add and move)

Load	R2, M(α)	2 to 3 cycles
Load	R3, M (β)	2 cycles due to overlapping
Add	R0, R1	2 cycles
Move	R1, R5	1 cycle
Multiply	R2, R3	3 cycles

Through this code rearrangement, the data dependences and program schematics are preserved, and the multiply can be initiated without delay

While the operands are being loaded from memory cells a and b into registers R_2 and R_3, the two instructions add and move consume three cycles and thus pipeline stalling is avoided

Static Scheduling

Hazard detection is conducted to a fixed point in the pipeline usually at the instruction decode stage

Disadvantage - all dependencies or collisions cannot be resolved in advance and hence potential hazards have to be treated as actual hazards

Register Scoreboarding

A separate register that has one bit to identify each register of the register file is called as a scoreboard $\Rightarrow$ bit is set if the corresponding register is in use

A computer with 32 registers will have a 32-bit score board to indicate the busy/free status of registers

When an instruction uses a set of registers, the bits corresponding to those registers are set to 1 in the scoreboard

They are reset to 0 as soon as the instruction using that register completes execution

The hardware will examine the scoreboard to see if the registers specified in the instruction are free before executing it

Scoreboarding

The scoreboard is a centralized control logic which keeps track of the status of registers and multiple functional units

When functional units generate new results, some data dependences can be resolved and thus a higher degree of parallelism can be explored with scoreboarding

Scoreboarding in latter microprocessors like MC 88000 used forwarding logic and register tagging ⇒ in a way, scoreboarding implements a kind of data-driven mechanism to achieve efficient computations

Dynamic instruction scheduling was implemented only in high-end mainframes or supercomputers in the past

Tomasulo's Algorithm

A hardware algorithm for dynamic scheduling of instructions that allows out-of-order execution and enables more efficient use of multiple execution units

Developed by Robert Tomasulo at IBM in 1967

This hardware dependence-resolution scheme was first implemented with multiple floating-point units of the IBM 360/91 processor

For Model 91 processor, three RSs were used in a floating-point adder and two pairs in a floating-point multiplier

The scheme resolve resource conflicts as well as data dependences using register tagging to allocate or deallocate the source and destination registers

Tomasulo's Algorithm

An issued instruction whose operands are not available is forwarded to an RS associated with the functional unit it will use

Waits until its data dependences have been resolved and operands become available

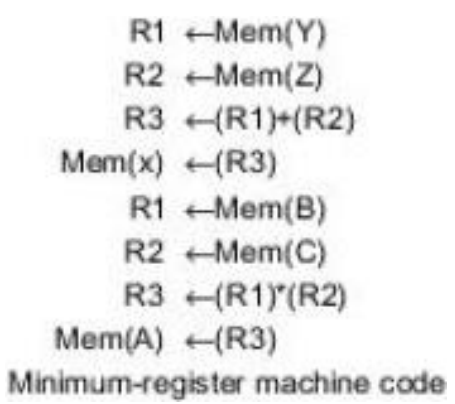
The dependence is resolved by monitoring the result bus

When all operands for an instruction are available, it is dispatched to the functional unit for execution

All working registers are tagged $\Rightarrow$ if a source register is busy when an instruction reaches the issue state, the tag for the source register is forwarded to an RS

When the register data becomes available, it also reaches the RS with same tag

Tomasulo's Algorithm: Example



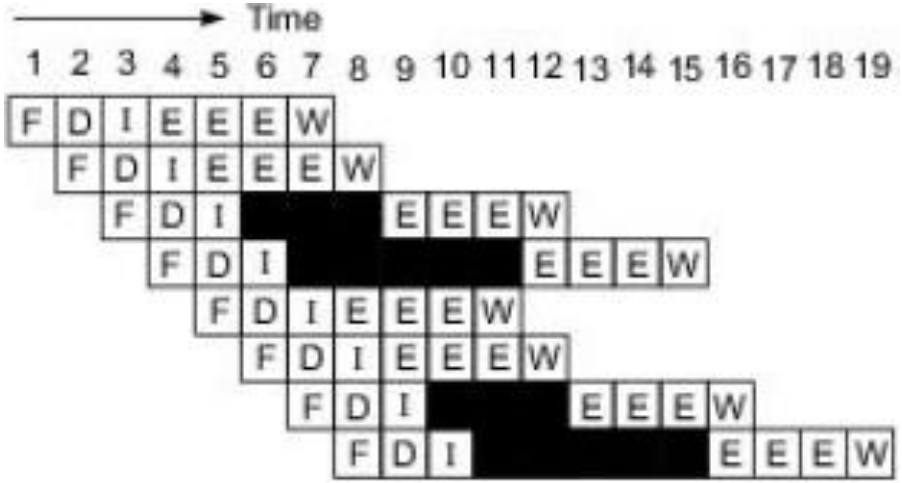
This algorithm was applied to work with processors having a few floating-point registers

In the case of Model 91, only four registers were available

Example: minimum-register machine code for computing X = Y + Z and A = B x C

```
R1 ←Mem(Y)
R2 ←Mem(Z)
R3 ←(R1)+(R2)
Mem(x) ←(R3)
R1 ←Mem(B)
R2 ←Mem(C)
R3 ←(R1)*(R2)
Mem(A) ←(R3)
```

Minimum-register machine code



The pipeline schedule

Dynamic instruction scheduling using Tomasulo's algorithm

Vector Processing

Vector & Vector Processing

Vector - an ordered set of scalar data items, all of the same type, stored in memory

Vector processor - an ensemble of hardware resources, including vector registers, functional pipelines, processing elements, and register counters, for performing vector operations

Vector processing - when arithmetic or logical operations are applied to vectors

Distinguished from scalar processing which operates on one or one pair of data

The conversion from scalar code to vector code $\Rightarrow$ vectorization

Vector processing is faster and more efficient

Vector Processing

Vector processing ⇒ a computer method that can process numerous data components at once

Operates on every element of the entire vector in one operation

Processor architecture that implements instructions operating on an array of data, aiming to enhance computing performance for data-intensive numerical simulations

Vector processor or array processor is a central processing unit (CPU) that implements an instruction set where its instructions are designed to operate efficiently and effectively on large one-dimensional arrays of data called vectors ⇒ this is in contrast to scalar processors, whose instructions operate on single data items only

Vector and Array Processors

Basic principle of vector and array processor $\Rightarrow$ only one instruction stream is there, that is monitored by a single control unit, and also a single program counter (PC) is processed by the system

They are both member groups of the supercomputer class and parallesism in them is of inter-instruction type

They are both designed to execute a common class of computation problems involving array of data or vectors

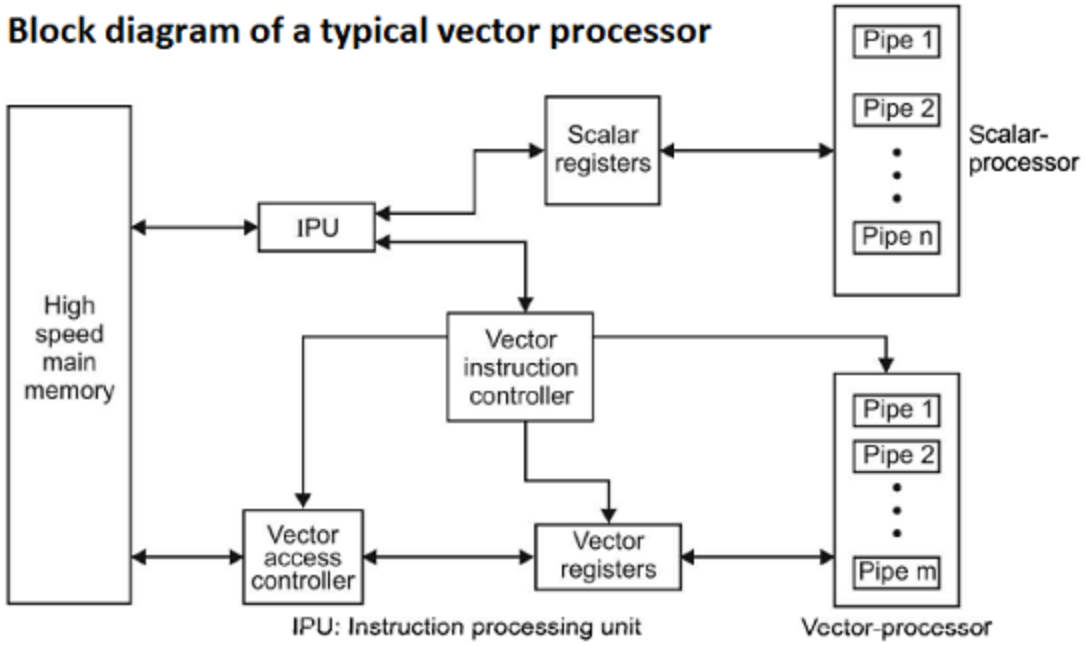
Both use data parallelism

Basic Vector Architecture

Architecturally, a typical vector processor consists of multiple functional pipes

A parallel task-scheduling model is presented for multi-pipeline vector processors

Figure shows the functional diagram of multiple-pipeline vector computer



Vector Processing Terminologies

Vector: a set of scalar data items, all of the same type, stored in memory

Stride (skew distance): vector elements are ordered to have a fixed addressing increment between successive elements called as stride or skew distance

Vector processor: an ensemble of hardware resources including vector registers, functional pipelines, PEs and register counters, for performing vector operations

Vector Processing: occurs when arithmetic or logical operations are applied to vectors

Vectorization: process of conversion for the scalar code to the vector code

Vectorizing compiler (Vectorizer): compiler capable of vectorization

Flushing time: time between the decoding of a vector instruction and the exit of the first result from the pipeline

Vector Instruction Types

Vector Instruction Types

Vector-vector instructions

Vector-scalar instructions

Vector-memory instructions

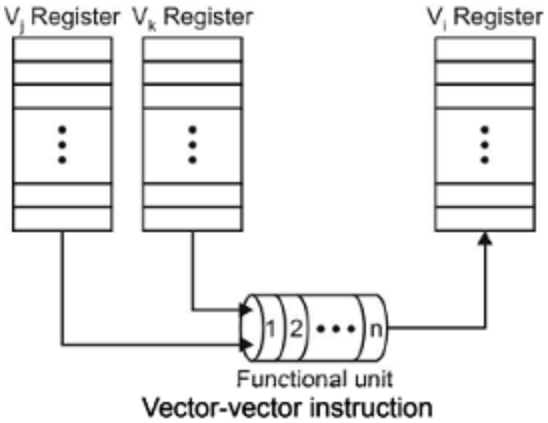
Vector-reduction instructions

Gather and scatter instructions

Masking instructions

Chaining

Vector-vector instructions



One or two vector operands are fetched from respective vector registers, enter through a functional pipeline unit and produce results in another vector register

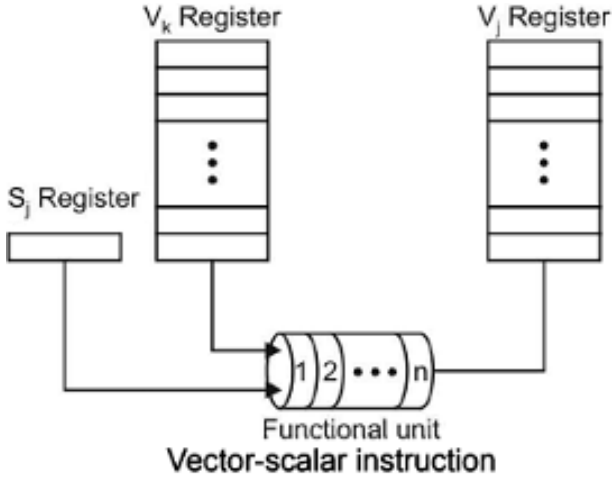
For example

$$f_1 : V_i \rightarrow V_j$$
$$f_2 : V_j \times V_k \rightarrow V_i$$

$V_1 = \sin(V_2)$ and $V_3 = V_1 + V_2$

for the mappings of above functions f_1 and f_2, where V_i for i = 1, 2 and 3 are vector registers

Vector-scalar instructions



Vector-scalar combination ⇒ sj and Vk register

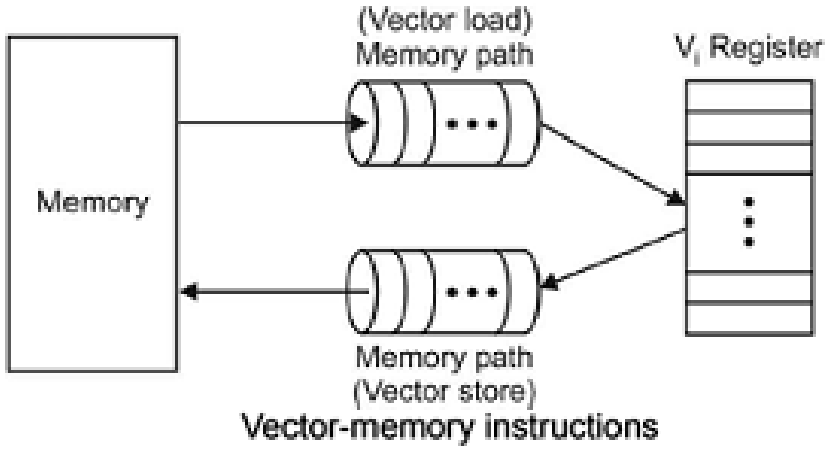
The operand is fetched and sent to the vector register

$$f_3 : S \times V_i \rightarrow V_j$$

Here, a scalar product S * V_1 = V_2

The elements of V_1 are each multiplied by a scalar S to produce vector V_2 of equal length

Vector-memory instructions



There is a vector load or vector store, element by element, between vector register (V) and memory (M)

$$f_4 : M \rightarrow V \text{ (Vector load)}$$
$$f_5 : V \rightarrow M \text{ (Vector store)}$$

Vector-reduction instructions

Let us see the mapping:

$$f_6 : V_i \rightarrow S_j$$
$$f_7 : V_i \times V_j \rightarrow S_k$$

An example of mapping f6 is finding maximum, minimum, sum and mean value of all elements in a vector

Similarly, an example of mapping f_7 is the dot-product that performs:

$$S = \sum_{i=1}^{n} a_i \times b_i \text{ (from two vectors)}$$

Gather and scatter instructions

These instructions use two vector registers to gather or to scatter vector elements randomly throughout the memory

$$f_8 : M \rightarrow V_1 \times V_0 \text{ (Gather)}$$
$$f_9 : V_1 \times V_0 \rightarrow M \text{ (Scatter)}$$

Gather ⇒ an operation that fetches from memory, the non-zero elements of a sparse vector using indices that themselves are indexed

Scatter ⇒ a reverse operations, that stores into memory, a vector in a sparse vector whose non-zero entries are indexed (a matrix in which most of the entries are zeros ⇒ sparse matrix)

Masking instructions

Use a mask (vector mask) to compress or to expand a vector to a shorter or longer index vector, respectively i.e.,

$$f_{10} : V_0 \times V_m \rightarrow V_1$$

Chaining

A technique used in some vector processors in which the output of one pipeline is directly released into another pipeline ⇒ enhance the performance of processors, and the time delays due to storing of intermediate results can be eliminated

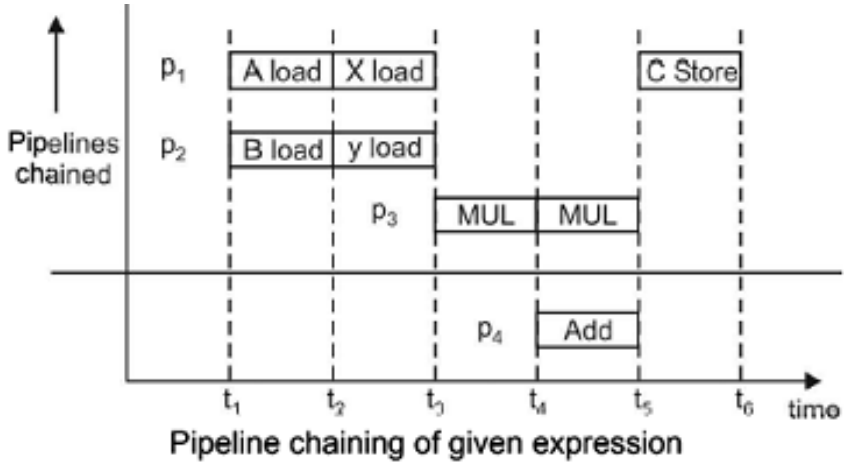
An intelligent compiler should have a capability of detecting sequences of operations that can be chained together

For e.g.,

$$C(I) = [A(I) \times B(I)] + [X(I) \times Y(I)]$$
$$\text{for } I \leftarrow 1, 2 \ldots\ldots N$$

Chaining can help here ⇒ this is how it helps

Say we have two load-store pipelines



Pipeline chaining of given expression

Vector Performance Modeling

Vector Performance Modeling

Performance of a pipelined vector processor is measured on the basis of following parameters:

- Speed-up (S_k)
- Utilization efficiency (η)
- Throughput (W)
- Performance-to-cost ratio (PCR)

To explain these parameters, let us assume certain notations

- k : The number of pipeline stages.
- T : The total pipeline delay in one instruction execution.
- n : The number of instruction contained in a task.
- N_i : The length of vector operands used in the ith instruction $(1 \leq i \leq n)$.
- W : The throughput of a pipeline computer.
- T_i : The time required to finish the ith instruction is a pipeline computer $(1 \leq i \leq n)$.
- T_p : The total time required to finish a task consisting of n–instructions.
- S_k : The speed–up of a pipeline computer with k stages. [over a serial computer].
- η : The efficiency of a pipeline computer.

Here in, if $N_i = 1$ means scalar instruction
& $N_i > 1$ means vector instruction

Speed-up

The time required per stage,

$$\tau = T/K$$

For a single vector instruction with vector length, N_i, the delay is:

$$\begin{aligned} T_i &= (k-1)\cdot\tau + N_i\cdot\tau \\ &= (N_i + k - 1)\cdot\frac{T}{K} \end{aligned}$$

Where $(k-1)\tau$ is the time required to fill up the pipe when N_i become long

When k is small,

$$T_i = N_i\cdot\tau$$

Now, say there is a sequence of n-vector instructions

Suppose that the execution of different types of vector instructions takes the same amount of time, if they have same vector length

So, total execution time required in a pipeline processor $= T_p = \sum_{i=1}^{n} T_i$

Finally we get, $$S_k = \frac{K \cdot \sum_{i=1}^{n} N_i}{(K-1)n + \sum_{i=1}^{n} N_i}$$

So, we can achieve a maximum possible speed-up if long vectors are processed in the pipeline

Utilization Efficiency (η)

Efficiency of the pipeline computer is defined as the total space-time product required by the job divided by a total available space-time product

$$\eta = S_K/K$$

So, efficiency (η) of a pipeline is the ratio of actual speed-up to the maximum possible speed-up K

The efficiency of the pipeline depends on the vector length distribution i.e., too many scalar operations of different types will downgrade the performance

Throughput (W)

Maximum throughput ⇒ the results that can be generated per unit time

Throughput rate reflects the processing capability of a pipeline processor

Let,

T_s: total time required to process a job in a non-pipelined, serial processor

K: pipeline stages

For a pipelined processor with K-stages, the clock period, τ should be

$$\tau = T_s / K + \theta$$

Where, τ : clock period for pipeline

θ : Latch delay

Neglecting the pipeline filling time ⇒ one instruction will flow out of the pipe at every τ units of time

Total time required for processing n-instruction = $n.\tau$

$$= T_s + n \cdot \theta$$

$$\text{Throughput}, \quad W = \frac{n}{n\tau} = \frac{n}{n\left(\frac{T_s}{K} + \theta\right)}$$

$$\boxed{W = \frac{1}{\left(\frac{T_s}{K} + \theta\right)}}$$

So, higher throughput may be obtained at the cost of more hardware

$$\eta = \frac{n\sum_{i=1}^{k} C_i \cdot \tau_i}{\sum_{i=1}^{k} C_i \cdot \left[\sum_{i=1}^{k} \tau_i + (n-1)\tau\right]}$$

Where

C_i = Cost of ith stage.

τ_i = Delay of ith stage.

k = stages

n = Total number of jobs streamed into the pipeline during the period of measurement

Performance-to-cost ratio (PCR)

In case of vector processors,

$$PCR = \frac{W}{C + kd} = \frac{1}{\left(\frac{T_s}{N} + \theta\right)(C + KD)}$$

Where, d: Average latch cost

C + kd: cost of entire pipeline

The optimal design of the static linear pipeline processor require k_0 stages such that PCR is maximized

$$i.e., \quad \frac{\partial(PCR)}{\partial k} = \frac{T_s \cdot \frac{C}{k^2} - \theta d}{\left(\frac{T_s}{k} + \theta\right)^2 \cdot (C + kd)^2}$$

$$\text{when} \quad \frac{\partial(PCR)}{\partial k} = 0$$

PCR

$$\text{then} \quad T_s \cdot \frac{C}{k^2} - \theta d = 0 \ ;$$

$$\text{At} \quad k = k_0,$$

$$T_s \cdot \frac{C}{k^2} - \theta d = T_s \cdot \frac{C}{k_0} - \theta d = 0$$

$$\text{or} \quad T_s \cdot \frac{C}{k^2} - \theta d = 0$$

$$\text{or} \quad k_0 = \sqrt{\frac{T_s \cdot C}{\theta d}}$$

Which is **maximum**

Vectorization

Vectorization

Vectorizing compilers / vectorizers are used for vectorization process

Vectorizing compiler analyzes Do-Loop in a high level language, that can be executed in parallel and generates object codes with vector instructions

The performance will be proportional to the vectorization ratio ⇒ i.e., the higher is the vectorization ratio, the higher is the performance

To achieve this, the compiler replaces a block of complicated data access and restructures the program sequence

Some barriers to vectorization exist in conditional and branch statements

Vectorization

In vector processing, we have to identify the Do-Loops or the segments of the program which are SIMD in nature and allocate them to various functional units simultaneously ⇒ Vectorization process

The efficient allocation ⇒ Optimization

The performance of vector processor depends on the vectorization and optimization

Vectorizing Compiler Design

Vectorizing Compiler Design

Vectorizing compiler (vectorizer) $\Rightarrow$ has the ability of detecting parallelism in serially coded programs

Recognizes the constructs that can be executed in parallel

Vectorizer(s) basically performs precedence analysis and code generation

An ideal vectorizer performs analysis of data dependencies and determines the possibility of vectorization

Vectorizing Compiler Design

An effective vectorizer should be able to

- Determine the flow pattern between subprograms or Do-Loops
- Check the precedence relationship among the subprograms
- Check the locality of variables
- Determine the loop variables
- Check the independence of variables
- Replace the inner loop with vector instructions

Vectorizer

Post compilation software which after going through the compiled code of the user program, detects the independent Do-Loops with the help of various dependent analysis algorithms and generate strip-mining code with the allocation of vector registers

e.g., for-loop of C

```
for (i=0; i<n; i++)
    C[i] = A[i] * B[i]
```

The size of vectors A, B and C is not predefined as the size will depend on the value of n, in given for-loop ⇒ vectorization become a problem

Vectorization Problems & Solutions

Vector's length can be controlled:

A vector length register (VLR) can be created that will control the length of vector operation

If we define values of VLR such that **VLR < length of the vector** then such Do-Loops / For-Loops can be vectorized until the real vector length is less than the maximum vector length ⇒ this is called as strip-mining

But it may have some problems like large startups which degrade the performance

Vectorization Problems & Solutions

Vector strides:

It can be used when the elements of vectors are not adjacent

Stride is the distance which separates the elements $\Rightarrow$ value of stride could be different for different variables

Stride is also known as a skew distance

When a vector is loaded into a vector register then the stride is one meaning that all the elements of vector are adjacent

Examples

Vectorize the following FORTRAN code:

```
      DO 100 I = 1, N
      A(I) = B(I) + C(I)
100   B(I) = 2 × A(I + 2) ?
```

Solution: Its vector form is:

```
A(1 : N) = B(1 : N) + C(1 : N)
TEMP (1 : N) = A(3 : N + 2)
B(1, N) = 2 × TEMP (1 : N)
```

Structures and Algorithms of Array Processors (SIMD Computers)

Structures and Algorithms of Array Processors (SIMD Computers)

SIMD Array Processors

SIMD Interconnection networks

Parallel Algorithms for Array Processors

Algorithm examples - matrix multiplication

Array Processor

A processor that is used to perform different computations on a huge array of data ⇒ called an array processor

Also called as vector processors or multiprocessors

Performs only single instruction at a time on an array of data

Works with huge data sets to execute computations

Mainly used for enhancing the performance of computers

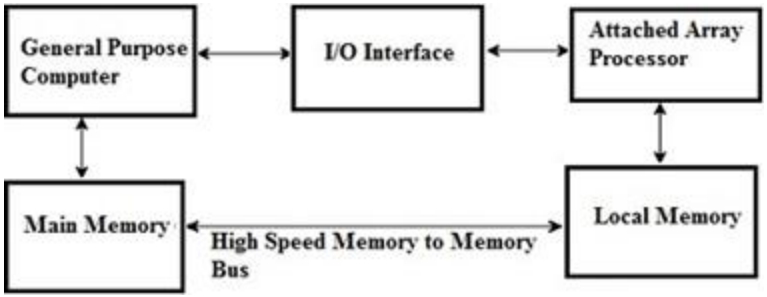
Its architecture mainly designed for processing arrays of numbers

This architecture contains a number of processors that works simultaneously, each handling one array element, so that a single operation is applied to all the array elements in parallel

Types of Array Processors

Two types of Array Processors

- Attached Array Processor, and
- SIMD Array Processor



Attached Array Processor Structure

SIMD Array Processor

SIMD is a computer with multiple processing units operating in parallel

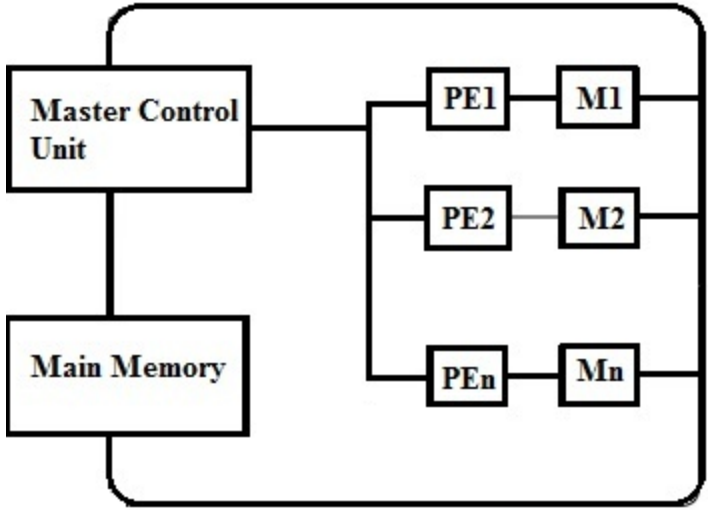
The processing units are synchronized to perform the same operation under the control of a common control unit

Provides a single instruction stream, multiple data stream (SIMD) organization

SIMD is a computers with several processing units which operate in parallel

These processing units perform the same operation in synchronizing under the supervision of the common control unit

The SIMD processor includes a set of identical PEs (processing elements) where each PES has a local memory



SIMD Array Processor Architecture

SIMD is a computer with multiple processing units operating in parallel

The processing units are synchronized to perform the same operation under the control of a common control unit

Thus providing a single instruction stream, multiple data stream (SIMD) organization

SIMD contains a set of identical processing elements (PES) each having a local memory M

SIMD Array Processors

A synchronous array of parallel processors is called an array processor

It consists of multiple processing elements (PEs) under the supervision of one control unit (CU)

An array processor can handle single instruction and multiple data (SIMD) streams

Array processors are also known as SIMD computers

SIMD machines are especially designed to perform vector computations over matrices or array of data

SIMD Interconnection Networks

SIMD array processor is mainly characterized by data routing network used in interconnecting the PEs

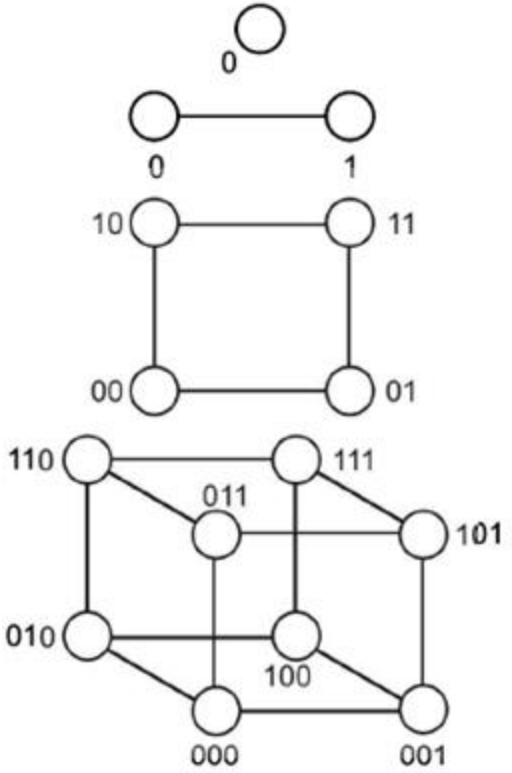
These networks are specified by data routing functions (DRFs)

This routing function is executed via interconnection network

Hypercube Interconnection Networks

Hypercube architecture can be of different dimensions

Various hypercubes with different dimensions are shown



dimension = 0, $n = 1$

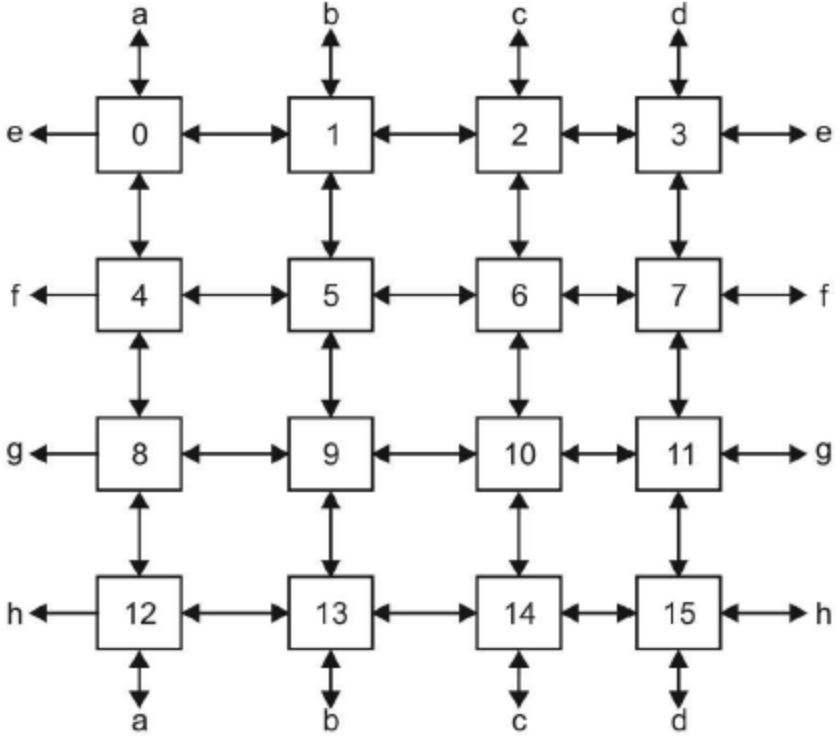
dimension = 1, $n = 2$

dimension = 2, $n = 4$

dimension = 3, $n = 8$

Hypercubes with dimensions 0, 1, 2 and 3

Mesh Connected Network



Parallel Algorithms for SIMD

Parallel Algorithms for SIMD

Parallel algorithm $\Rightarrow$ defines how a given problem can be solved on a given parallel computer

- How the problem can be divided into sub problems
- How the processors communicate
- How the partial solutions are combined to produce the final result

Parallel Algorithms Example

Matrix multiplication

Sequential Algorithms

Classical algorithm for matrix multiplication is:

```
for  (i = 0 ; i < n ;  i++)
     for (j = 0 ;  j < n ;  j++)
     {   C [i] [j] = 0 ;
        for (k = 0 ; k < n ;  k++)
           C[i] [j] = C[i] [j] + a [i] [k] *  b [k] [j];
     }
```

Analysis of algorithm:

- Requires n^3 multiplications and n^3 additions
- Time complexity = $O(n^3)$

Parallel Algorithm Using mesh implementation

Most natural architecture for matrix operations is a 2D mesh

In 2D mesh, each node computes one element of the result array

Mesh connection enables message to pass between adjacent nodes in the mesh simultaneously

Two methods of matrix multiplication for a mesh organization:

- Cannon's algorithm
- Systolic approach

Cannon’s Algorithm

Uses a mesh of processors with wraparound connection to shift the A elements left & the B elements up

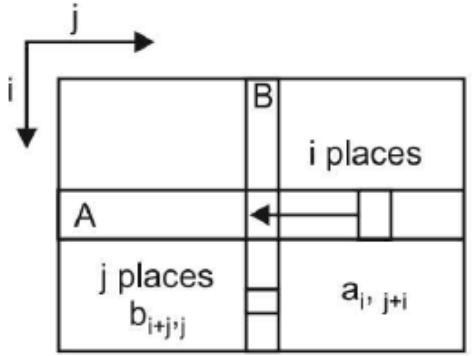
All shifts are with wraparound

Steps

S1: Initially processor $P_{i,j}$ has elements $a_{i,j}$ and $b_{i,j}$ ($0 \leq i < n$, $0 \leq j < n$)

Cannon’s Algorithm

S2: Elements are moved from their initial position to an 'aligned' operation. The complete i^{th} row of A is shifted i places left & complete j^{th} column of B is shifted j places upward. This has the effect of placing the elements $a_{i,j+i}$ and the element $b_{i+j,j}$ in processor $P_{i,j}$. These elements are a pair of those required in accumulation of $c_{i,j}$

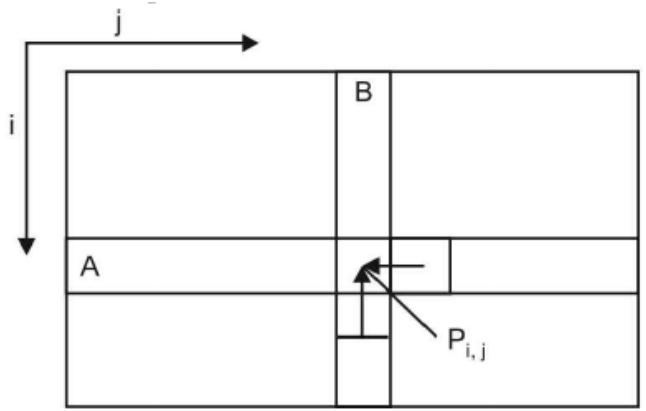


S2: Alignments of elements of A and B

Cannon's Algorithm

S3: Each processor, $p_{i,j}$ multiplies its elements

S4: The ith row of A is shifted one place left, & the jth of B is shifted one place upward. This has the effect of bringing together the adjacent element of A and B, which will also be required in the accumulation



S4: One place shift of elements A and B

Cannon's Algorithm

S5: Each processor, $p_{i,j}$ multiplies the elements brought to it & adds the results to the accumulating sum

S6: Steps 4 & 5 are repeated until final result is obtained (n-1shifts with n rows & n columns of elements)

Additional storage for partial results is unnecessary with this algorithm

Analysis: Each sub matrix multiplication requires m^3 multiplication and m^3 additions $\Rightarrow$ hence with S-1 shifts

$$t_{comp} = 2s\ m^3 = 2m^2\ n$$

Or time complexity is $O(m^2\ n)$

Systolic approach

Systolic - meaning of heart pumps blood in medical field

Information is pumped through a systolic array in various directions and at regular intervals

2D systolic array $\Rightarrow$ information is pumped from left to right and from top to bottom

Information meets at internal nodes where processing occurs

Information passes onward i.e. left to right or downward

Systolic approach

The figure shows a systolic array used to multiply two (4x4) matrices, A & B

The elements of A enter from left and the elements of B enter from top

